



Линейное внимание

Введение в эффективные системы глубокого обучения

Алексеев Илья, СБЕР Риски @voorhs
Лекция 04, осень 2026

Раздел 0

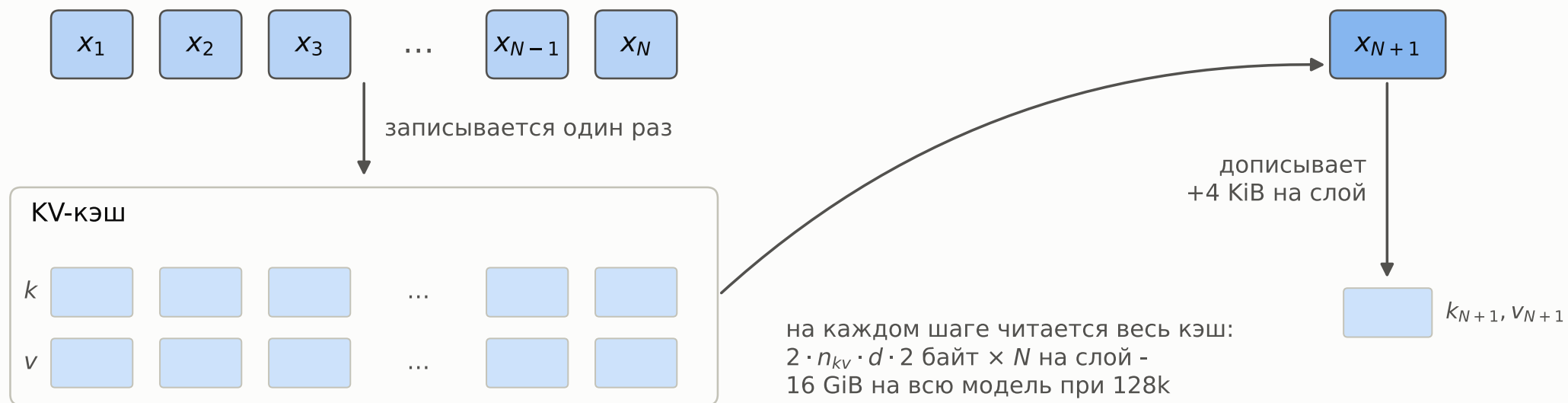
Арифметика KV-кэша

Префилл и декодирование

$$O = \text{softmax}(QK^{\top} + M) V$$

Префилл: $x_1 \dots x_N$ обрабатываются одним проходом
один пакет матричных умножений - доминирует арифметика

Декодирование: по одному токенту
арифметики почти нет - доминирует чтение KV-кэша



время шага $\propto$ размер кэша $\propto N$; тензорные ядра ждут память

KV-кэш одного слоя

$$2 \cdot n_{kv_heads} \cdot head_dim \cdot dtype_bytes \cdot N$$

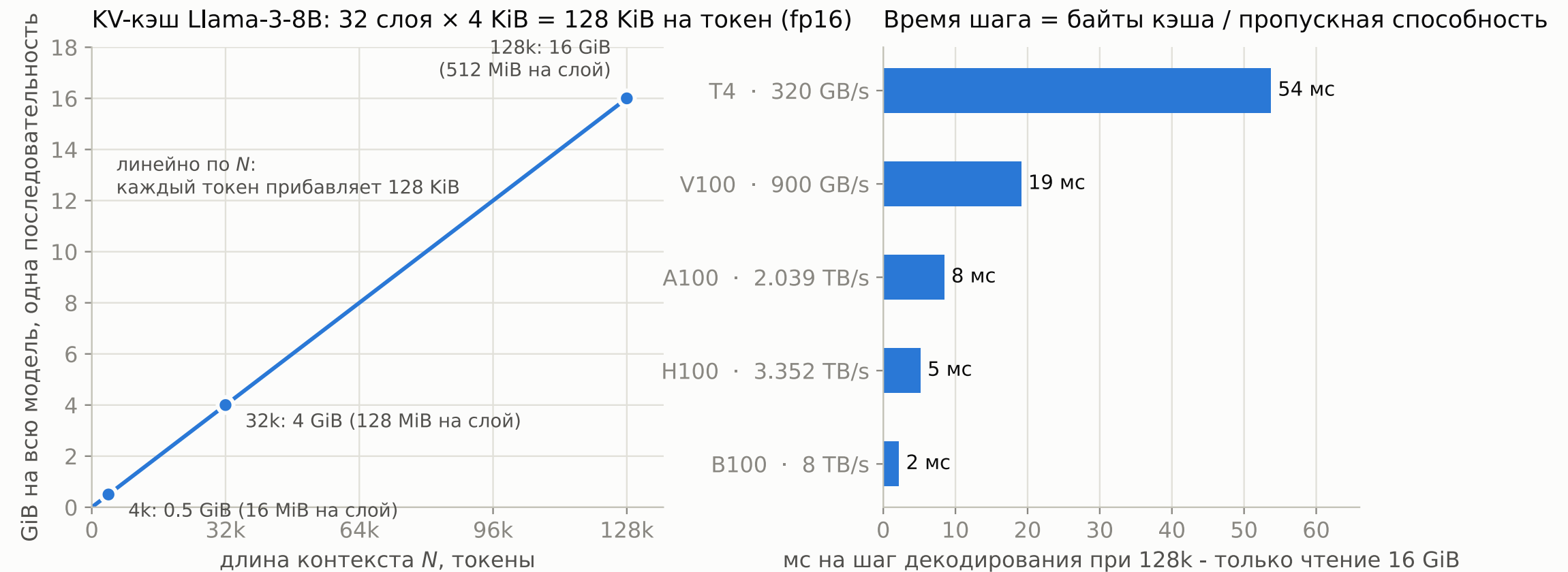
Llama-3-8B: 32 слоя · $n_heads = 32 \cdot n_{kv_heads} = 8 \cdot head_dim = 128 \cdot fp16$: 2 байта

$$2 \cdot 8 \cdot 128 \cdot 2 = 4096 \text{ байт на токен, на слой}$$

	один слой	32 слоя
на токен	4 KiB	128 KiB
128k токенов	512 MiB	16 GiB

одна последовательность, сверх весов

KV-кэш Llama-3-8B и время шага декодирования



Вопрос лекции

Как ограничить рост KV-кэша с увеличением контекста?

на доске

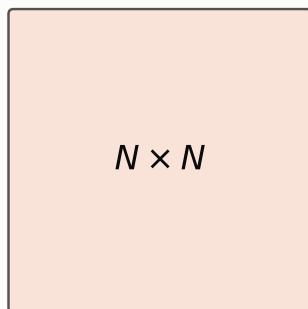
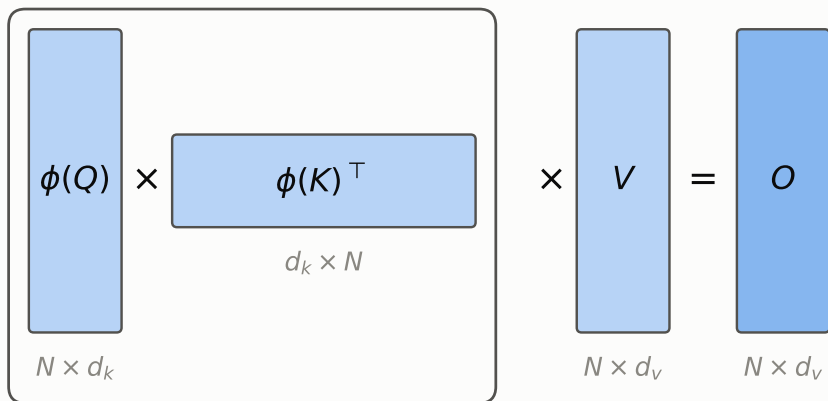
Раздел 1 · Линейное внимание и память

$$O = \text{softmax}(QK^{\top} + M) V$$

$$q, k \in \mathbb{R}^{d_k} \quad v \in \mathbb{R}^{d_v}$$

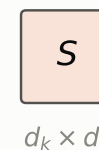
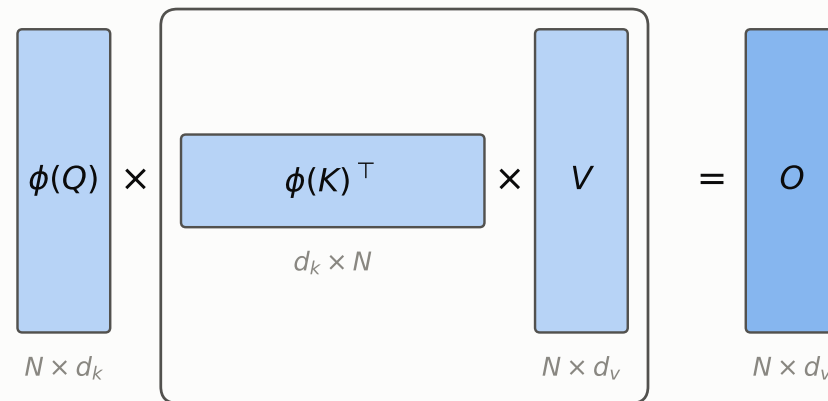
Две расстановки скобок

$$(\phi(Q) \phi(K)^\top) V$$



промежуточный объект:
матрица оценок
 $O(N^2 d)$ арифметики
 $O(N^2)$ памяти

$$\phi(Q) (\phi(K)^\top V)$$



промежуточный объект:
 $S = \phi(K)^\top V$, от N не зависит
 $O(N d_k d_v)$ арифметики
 $O(d_k d_v)$ памяти

S — вся память слоя о прошлом

$$S = \phi(K)^\top V, \quad S \in \mathbb{R}^{d_k \times d_v}$$

$32 \cdot 128 \cdot 128 \cdot 2 = 1 \text{ MiB}$ на слой: 32 головы, $d_k = d_v = 128$, 2 байта

	softmax-внимание	линейное внимание
байты состояния на слой при длине контекста N	512 MiB при 128k	1 MiB при любом N
прирост байтов на токен, на слой	4 KiB	0

×512 при 128k

Отображение признаков ϕ

$$\text{sim}(q, k) = \phi(q)^\top \phi(k),$$

ϕ

Linear Transformer

$\text{elu}(x) + 1$: положительно и дёшево

Performer

случайные признаки, приближение exp

современные модели

$\phi(x) = x$, L2-нормированные q и k , нормализация выхода

ϕ не меняет размер S , а узкое место — размер S

Раздел 2

Рекуррентная форма и интерференция

Рекуррентная форма

каузальность: сумма по префиксу, своё S в каждой позиции

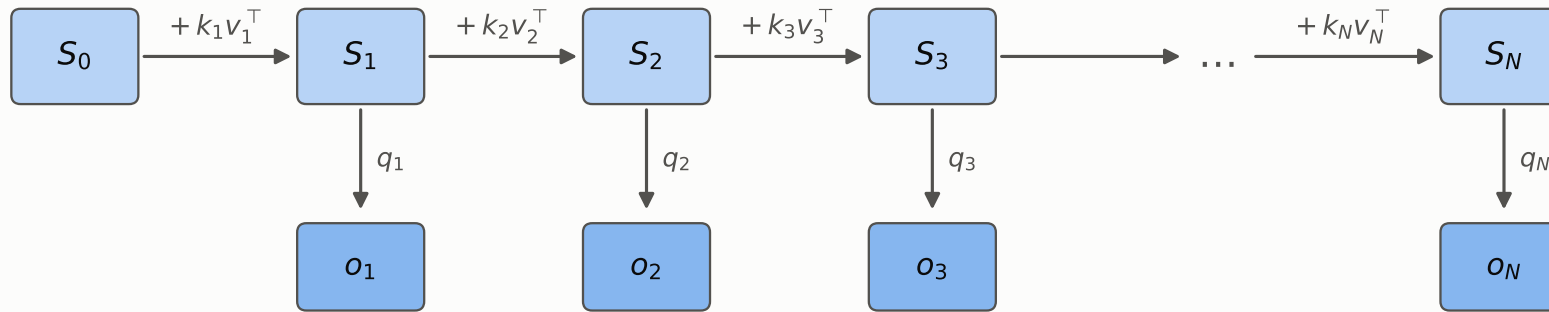
$$S_i = \sum_{j \leq i} k_j v_j^\top$$

$$S_t = S_{t-1} + k_t v_t^\top$$

$$o_t = S_t^\top q_t$$

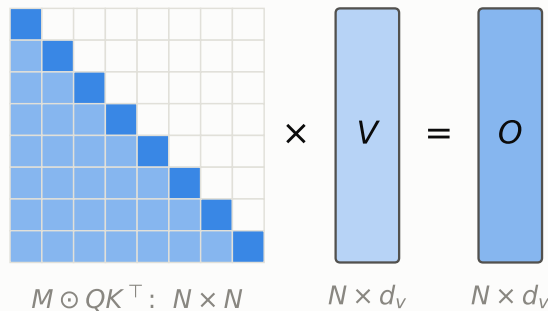
Рекуррентная и параллельная формы

Рекуррентная форма: $S_t = S_{t-1} + k_t v_t^\top$, $o_t = S_t^\top q_t$



одна и та же функция: все o_t совпадают

Параллельная форма: $O = (M \odot QK^\top) V$, $M = \text{tril}(\text{ones}(N, N))$



все N выходов разом - одно матричное произведение;
строка i суммирует по $j \leq i$

диагональ входит в маску: токен видит сам себя

■ $j = i$

Что мы построили

RNN с матричным состоянием

- состояние $S \in \mathbb{R}^{d_k \times d_v}$ вместо вектора
- шаг: $d_k d_v + d_k d_v$ умножений-сложений, от N не зависит

Хеббовская ассоциативная память

- запись пары (k, v) : $S \leftarrow S + kv^\top$
- чтение: $S^\top q$

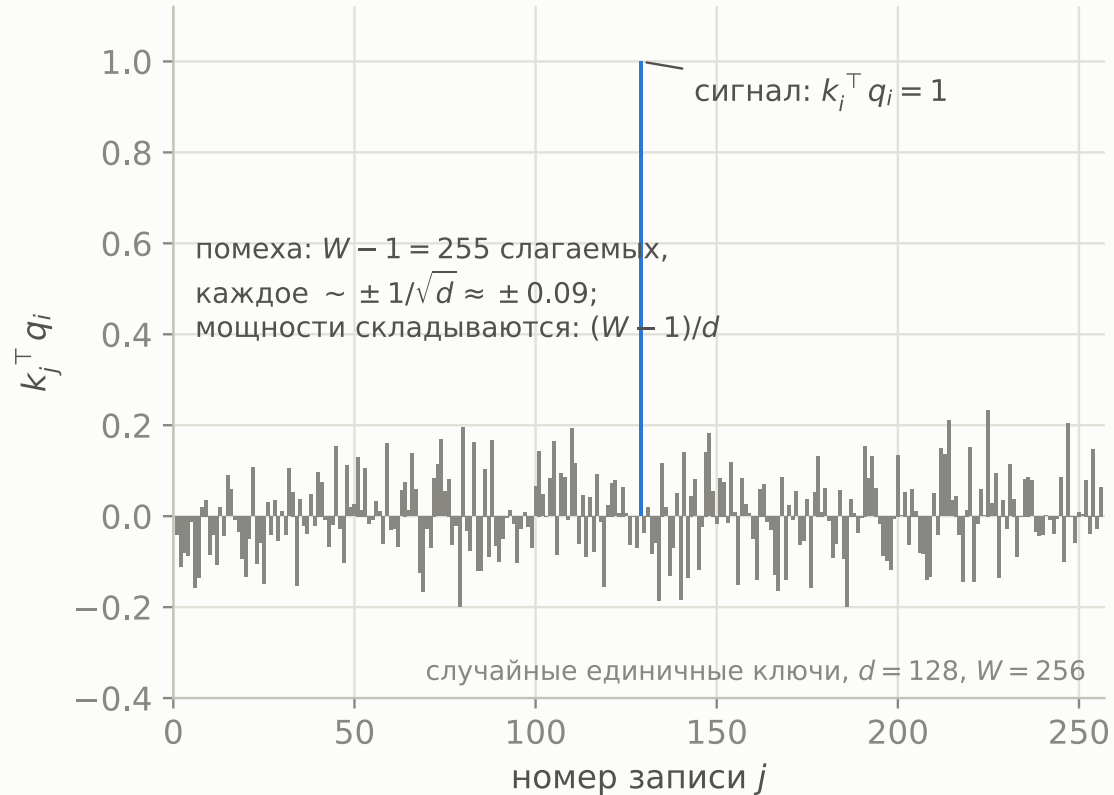
Разложение на сигнал и перекрёстные помехи

$$\begin{aligned} S_t^\top q_i &= \left(\sum_{j \leq t} k_j v_j^\top \right)^\top q_i \\ &= \sum_{j \leq t} (k_j v_j^\top)^\top q_i \\ &= \sum_{j \leq t} v_j k_j^\top q_i \\ &= \sum_{j \leq t} (k_j^\top q_i) v_j \end{aligned}$$

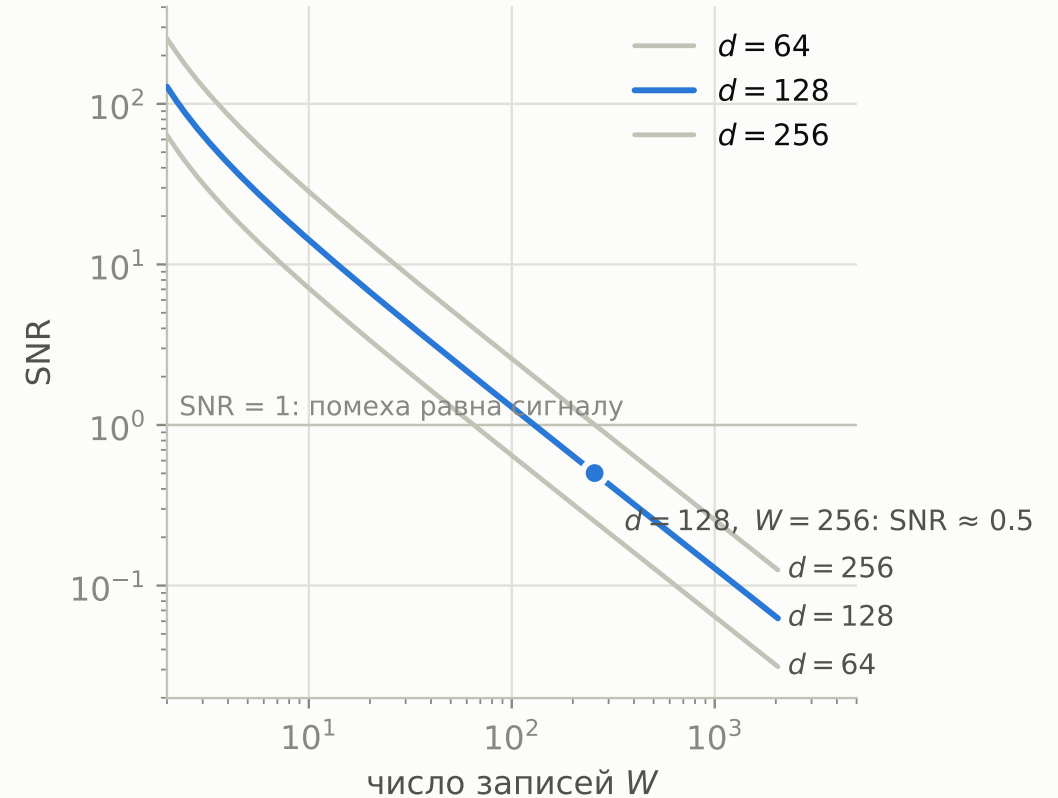
$$S_t^\top q_i = \underbrace{(k_i^\top q_i) v_i}_{\text{сигнал}} + \underbrace{\sum_{j \neq i} (k_j^\top q_i) v_j}_{\text{помеха}}$$

Сигнал и помеха при считывании

Коэффициенты считывания $k_j^T q_i$ при $q_i = k_i$: сигнал и помеха



Отношение сигнал/шум $\text{SNR} \approx d/(W - 1)$



Ёмкость

Ортонормированные ключи, $q_i = k_i$

- помеха = 0: воспроизведение точное
- таких ключей в $\mathbb{R}^{d_k}$ ровно d_k
- $d_k = 128$: не больше 128 ассоциаций на голову

Случайные единичные ключи

- $k_j^\top q_i$: среднее 0, дисперсия $1/d$
- W записей: мощность помехи $(W - 1)/d$, мощность сигнала 1

$$\text{SNR} \approx \frac{d}{W - 1}$$

$$\text{SNR} \approx \frac{128}{256 - 1} \approx 0.5 \quad d = 128, W = 256: \text{помеха вдвое больше сигнала}$$

Раздел 3

Правило обновления

$S_t = S_{t-1} + k_t v_t^\top \rightarrow$ 3.1. Гейт $\rightarrow$ 3.2. Дельта-правило $\rightarrow$ 3.3. Совмещаем оба $\rightarrow$ 3.4.
Нормализация и параметризация

Состояние насыщается

$$S_t = S_{t-1} + k_t v_t^\top$$

каждая запись — навсегда и с тем же весом: токен 4 слова назад и 40,000 слов назад

$$\text{SNR} \approx \frac{d}{W - 1}$$

W растёт с каждым токеном — SNR падает монотонно, без дна

3.1. Гейт

$$S_t = \alpha_t S_{t-1} + k_t v_t^\top$$

$\alpha_t < 1$

затухание старого состояния до записи

запись τ шагов назад

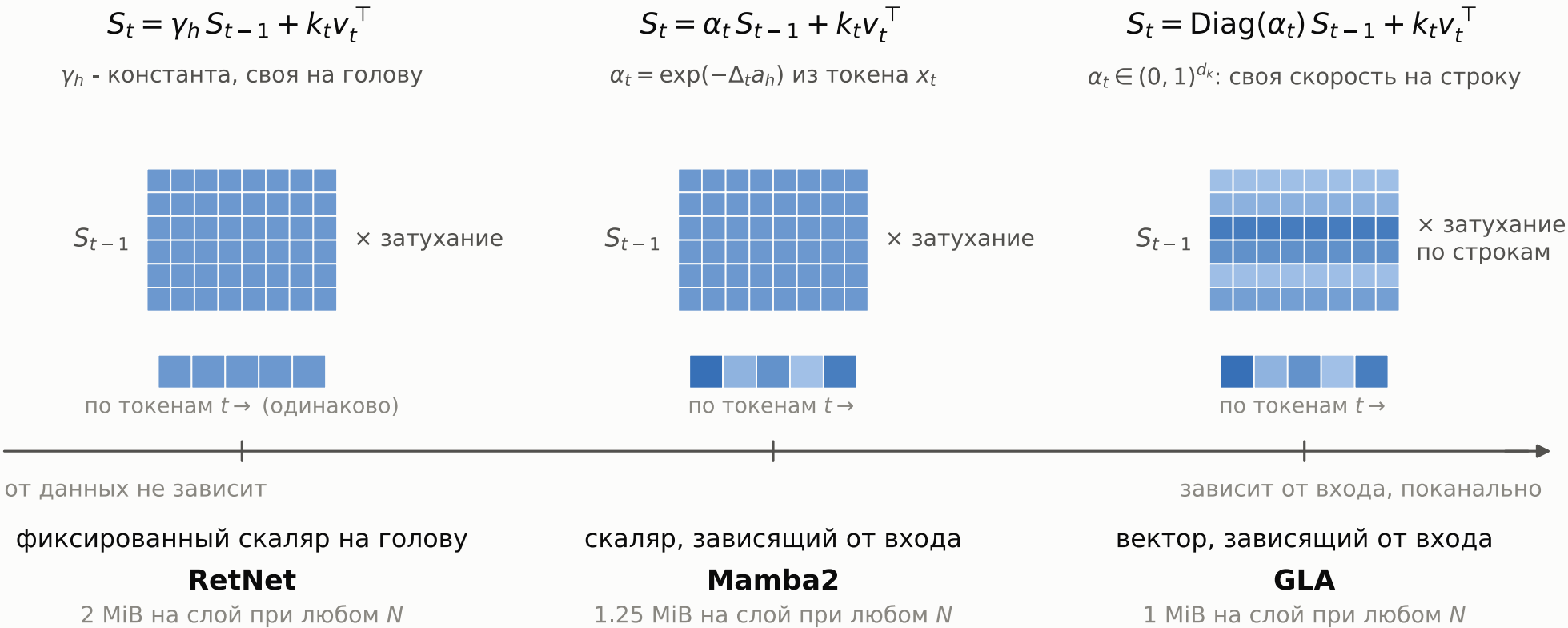
$\times \alpha_{t-\tau+1} \cdots \alpha_t$: влияние затухает геометрически

итог

эффективный горизонт вместо бесконечного

Три способа задать гейт

Три способа задать гейт забывания - одна ось, три архитектуры



3.2. Дельта-правило

запись (k, v_1) , позже (k, v_2) : тот же ключ, новое значение — «столица — Бонн», потом «столица — Берлин»

$$S = kv_1^\top + kv_2^\top = k(v_1 + v_2)^\top$$

$$\mathcal{L}_t(S) = \frac{1}{2} \|S^\top k_t - v_t\|^2$$

$$S \in \mathbb{R}^{d_k \times d_v} \quad k_t \in \mathbb{R}^{d_k} \quad v_t \in \mathbb{R}^{d_v} \quad \beta_t \in [0, 1] \text{ — шаг обучения}$$

Дельта-правило

$$S_t = (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top, \quad \beta_t \in [0, 1]$$

перезапись $\beta_t = 1, \|k_t\| = 1: I - k_t k_t^\top$ — проектор; чтение по k даёт v_2 , а не $v_1 + v_2$

стирающий множитель слева $I - \beta_t k_t k_t^\top: d_k \times d_k \cdot S_{t-1}: d_k \times d_v$

градиентный шаг $S_t = S_{t-1} - \beta_t \nabla_S \mathcal{L}_t(S_{t-1})$

последовательность — выборка, правило — оптимизатор, S — параметры; гейт — затухание весов (weight decay)

DeltaNet, 1.3B: 512 KiB на слой

3.3. Совмещаем оба: Gated DeltaNet

$$S_t = \alpha_t (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$$

α_t ослабить всё состояние: гейт, глобально

$I - \beta_t k_t k_t^\top$ стереть старую ассоциацию вдоль k_t : локально

$\beta_t k_t v_t^\top$ записать новую

Gated DeltaNet, 1.3B: 1.37 MiB на слой при любом N , прирост 0

Gated DeltaNet-2

$$S_t = (I - k_t(b_t \odot k_t)^\top) \text{Diag}(\alpha_t) S_{t-1} + k_t(w_t \odot v_t)^\top$$

$b_t \in [0, 1]^{d_k}$ — гейт стирания, $w_t \in [0, 1]^{d_v}$ — гейт записи: декаплинг стирания и записи

частный случай	правило	байты состояния на слой
общий	Gated DeltaNet-2	512 KiB (в статье fp32: 1 MiB)
$b_t = w_t = \beta_t \mathbf{1}$	KDA, Kimi Linear	1 MiB
и α_t — скаляр	Gated DeltaNet	1.37 MiB
без гейта	DeltaNet	512 KiB
без стирающего множителя	правило с гейтом	
без обоих	линейное правило	

прирост байтов на токен, на слой — 0 во всех строках

3.4. Нормализация и параметризация

Знаменатель $\rightarrow$ RMSNorm

- $q^\top \sum_j \phi(k_j)$ при $\phi(x) = x$: близок к нулю, меняет знак
- вместо деления — RMSNorm на выходе, иногда и выходной гейт

L2-нормировка q, k

- $\|k\| = 1$: собственные значения $I - \beta k k^\top$ в $[1 - \beta, 1]$, сжатие
- без нормировки: собственные значения ниже -1

Гейты

- β_t : сигмоида от линейной проекции токена
- $\alpha_t = \exp(-\exp(a) \text{softplus}(\cdot)) \in (0, 1)$

Короткая свёртка

- каузальная depthwise, ширина 3–4
- на q, k, v перед миксером
- DeltaNet, Gated DeltaNet, KDA, Qwen3-Next

Раздел 4

Чанковая форма

Почему ни одна из двух форм не годится для обучения

Параллельная форма

$$O = (M \odot (QK^{\top})) V, \quad QK^{\top} \in \mathbb{R}^{N \times N}$$

$N^2 d_k + N^2 d_v$ умножений-сложений: $O(N^2 d)$ по времени · N^2 чисел на голову: $O(N^2)$ по памяти

$$8 \cdot 16384^2 \cdot 4 = 8 \text{ GiB} \quad B \cdot H = 8, N = 16384, \text{ fp32: один тензор оценок}$$

оценки и их маскированная копия: 16 GiB, на практике около 17 GiB

Рекуррентная форма: арифметическая интенсивность

один шаг

$S \leftarrow S + kv^\top$	$d_k d_v$ умножений-сложений
----------------------------	------------------------------

$o = S^\top q$	$d_k d_v$ умножений-сложений
----------------	------------------------------

FLOP	$2 \cdot 2d_k d_v = 4d_k d_v$
------	-------------------------------

байты: прочитать и записать S в fp32	$2 \cdot 4d_k d_v = 8d_k d_v$
--	-------------------------------

арифметическая интенсивность $\frac{4d_k d_v}{8d_k d_v} = 0.5$ FLOP на байт, при любых d_k, d_v

баланс карты: $\approx 8 \cdot 10^{12}$ FLOP/s / ≈ 300 GB/s ≈ 25 FLOP на байт $\rightarrow$ в 50 раз ниже, $\leq 2\%$
арифметики $\cdot N$ последовательных шагов

Чанк

чанк длины $L \cdot S$ — память до чанка · $Q, K: L \times d_k \cdot V: L \times d_v \cdot j$ пробегает чанк

$$o_i = \underbrace{S^\top q_i}_{QS} + \underbrace{\sum_{j \leq i} (k_j^\top q_i) v_j}_{(M \odot QK^\top)V}$$

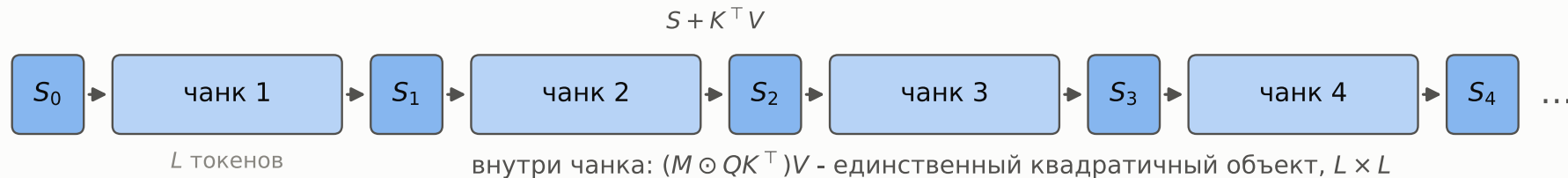
$$O = QS + (M \odot (QK^\top))V$$

$$S_{\text{out}} = S + K^\top V$$

внутри чанка — квадратично, но только $L \times L$ · между чанками — рекуррентность, N/L шагов

Два уравнения чанка с размерностями

Между чанками - рекуррентность по состоянию (N/L шагов), внутри чанка - матричные умножения



$$O = Q S + (M \odot QK^T) V$$

$L \times d_v$ $L \times d_k$ $d_k \times d_v$ $L \times L$ $L \times d_v$

QS : каждый токен чанка читает одно и то же входящее состояние своим запросом

$(M \odot QK^T)V$: вклад токенов чанка друг в друга, маска по $j \leq i$

четыре плотных матричных умножения на чанк - под них построено железо

$$S_{out} = S + K^T V$$

$d_k \times d_v$ $d_k \times d_v$ $d_k \times L$ $L \times d_v$

$K^T V$: сумма внешних произведений чанка одной матрицей -

это и делает переход между чанками одним шагом вместо L

Шаг по чанку: арифметическая интенсивность

на чанк, $d_k = d_v = d$

два произведения $L \times L$

$4L^2d$ FLOP

два произведения с состоянием

$4Ld^2$ FLOP

Q, K, V на вход, O на выход, S резидентно $4Ld$ чисел, $16Ld$ байт

$$\frac{4Ld(L + d)}{16Ld} = \frac{L + d}{4} \quad \text{FLOP на байт: растёт с } L \text{ и } d; \text{ у рекуррентности } 0.5$$

тензор оценок: L^2 на чанк, а не N^2 — при $L = 128$ и 16k токенов в 16,000 раз меньше

Стоимость и оптимальный размер чанка

$$O(NLd + Nd_k d_v)$$

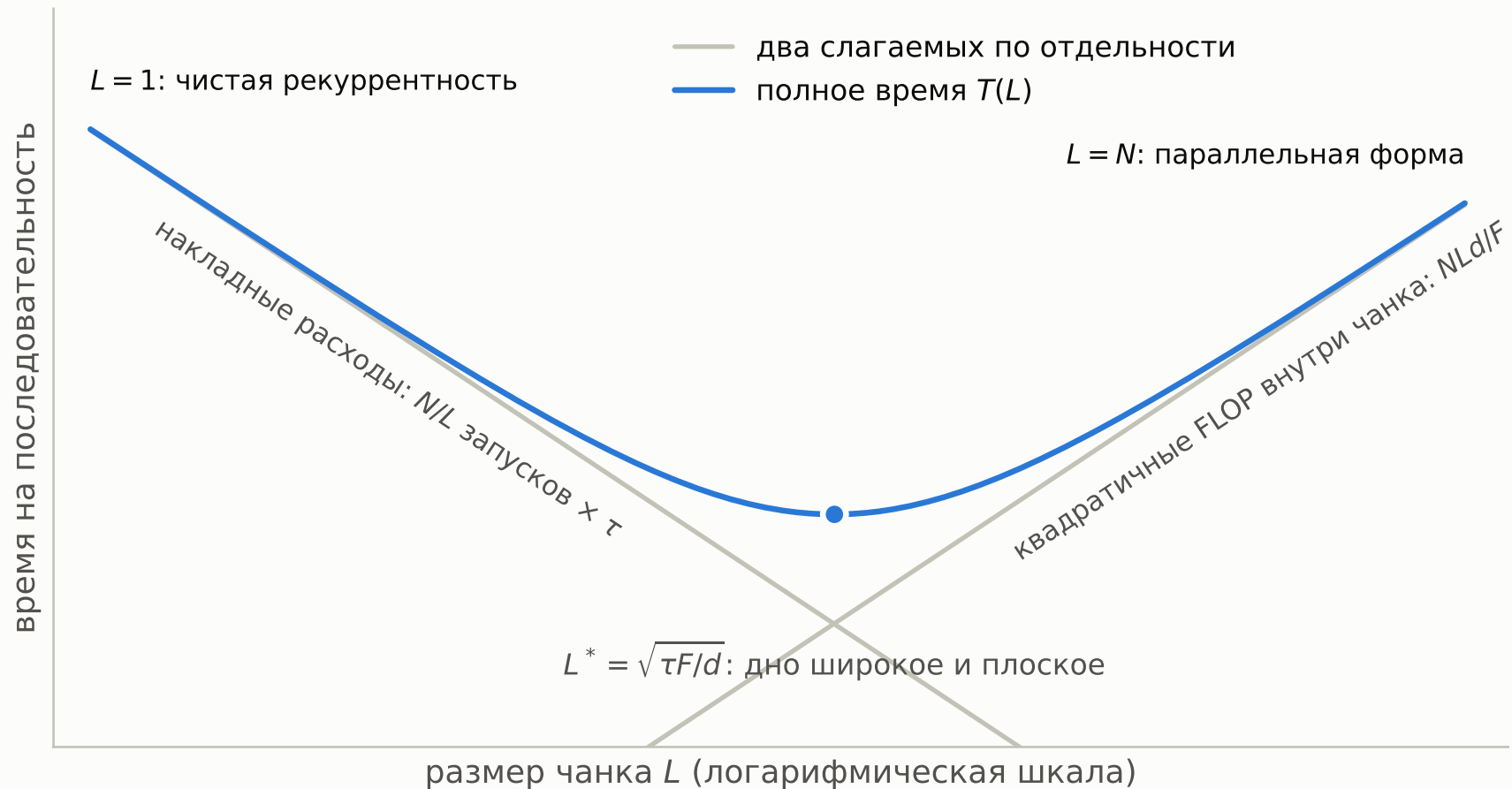
$$T(L) = \frac{N}{L} \tau + \frac{NLd + Nd^2}{F}$$

$$\frac{dT}{dL} = -\frac{N\tau}{L^2} + \frac{Nd}{F} = 0 \quad \implies \quad L^* = \sqrt{\frac{\tau F}{d}}$$

τ — накладные расходы на чанк · F — достигнутая скорость · L^* от N не зависит

U-кривая

U-кривая: $T(L) = \frac{N}{L} \tau + \frac{NLd + Nd^2}{F}$ - сообщение в форме, не в числах



τ : цикл Python и запуски ядер, в слитном ядре почти исчезает · предсказуемы диапазон и причина

С DeltaNet сложнее

линейное правило: сумма схлопывается — $S_{\text{out}} = S + K^{\top} V$

$$(I - \beta_L k_L k_L^{\top}) \cdots (I - \beta_2 k_2 k_2^{\top}) (I - \beta_1 k_1 k_1^{\top})$$

дельта-правило: произведение не схлопывается → СЛАУ с треугольной матрицей

Yang, Wang, Zhang, Shen, Kim · arXiv 2406.06484

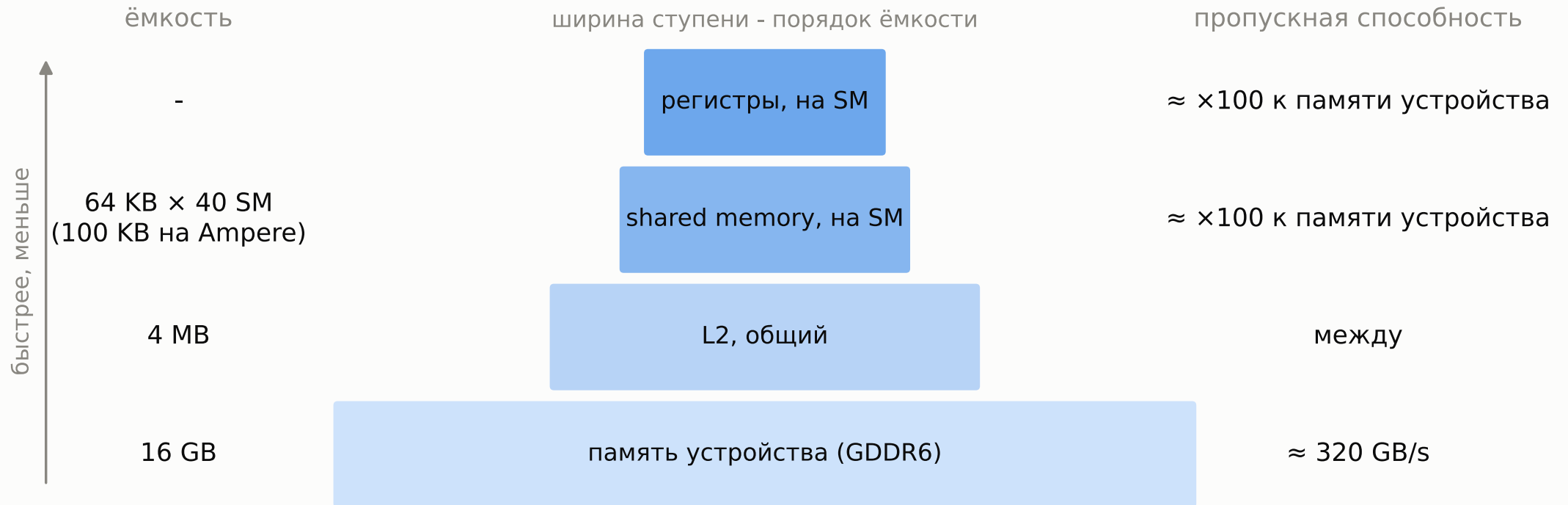
чанковый алгоритм — отдельный результат для каждого правила обновления

Раздел 5

Реальность железа

Иерархия памяти GPU

Иерархия памяти GPU - NVIDIA T4 (Colab, Kaggle); паспортные данные, не измерения



SRAM относительно памяти устройства: $\approx 10^2$ раз быстрее, $\approx 10^5$ - 10^6 раз меньше

Roofline-модель

π — пиковая производительность, FLOP/s · β — пропускная способность памяти, байт/s · I — арифметическая интенсивность, FLOP на байт

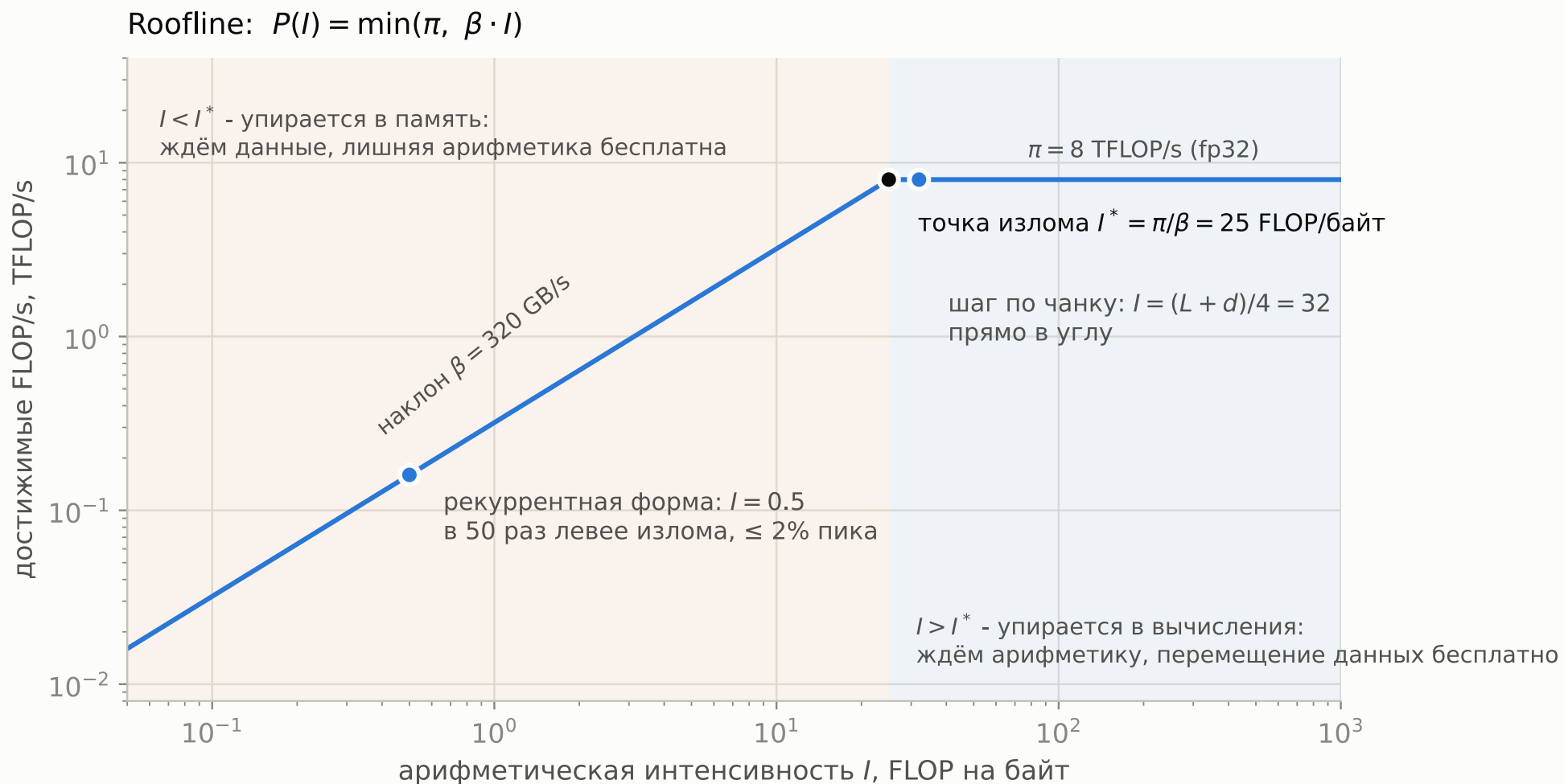
$$P(I) = \min(\pi, \beta \cdot I)$$

$$I^* = \frac{\pi}{\beta} \approx \frac{8 \cdot 10^{12}}{320 \cdot 10^9} = 25 \quad \text{FLOP на байт: точка излома (ridge point), T4}$$

$I < I^*$ упирается в память: лишняя арифметика бесплатна

$I > I^*$ упирается в вычисления: лишнее перемещение данных бесплатно

Две формы на roofline



рекуррентная форма: $I = 0.5$ · шаг по чанку: $I = (L + d)/4 = 32$ при $L = d = 64$

Слитное (fused) ядро

	наивная чанковая форма, PyTorch	слитное ядро
матричные умножения	отдельное ядро на каждое	одно ядро
Q, K, V	в память устройства и обратно между операциями	читаются ровно один раз
блок оценок	в память устройства и обратно	не покидает кристалл
состояние S	в память устройства и обратно	в регистрах всю последовательность; наружу дважды
O		записывается ровно один раз

та же идея, что в FlashAttention: алгоритм прежний, изменилось перемещение данных

Раздел 6

Работает ли это?

Перплексия и ассоциативная память

Языковое моделирование

- равные параметры и бюджет токенов
- разрыв по перплексии с трансформером мал

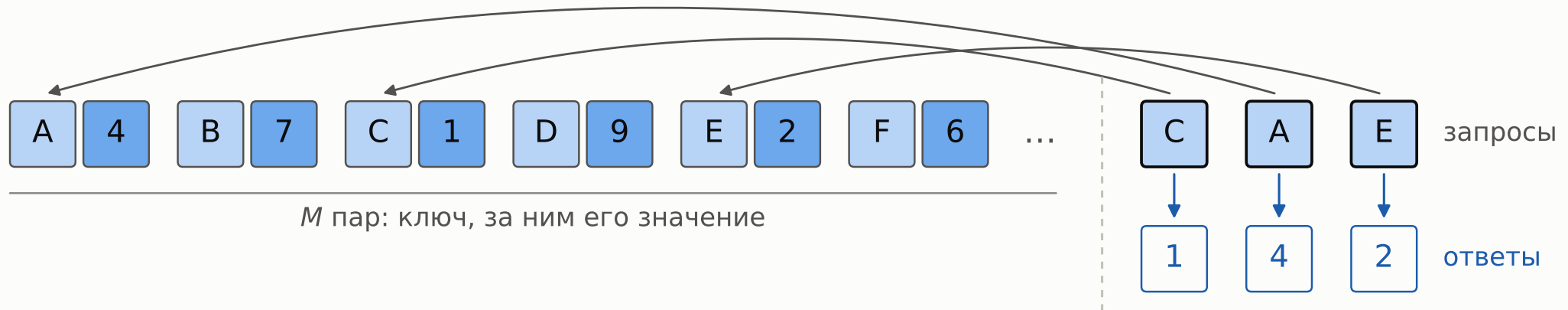
Ассоциативная память

- поиск иголки в стоге сена
- многошаговые обращения по длинному документу
- обучение в контексте на длинном списке примеров
- код со ссылками на тысячи токенов назад

отказ: не расплывчатый ответ, а уверенно извлечённое не то

MQAR

MQAR (multi-query associative recall): пары «ключ - значение», затем запросы по ключам



каждый токен пишет в состояние $+kv^T$; на запросе чтение $S^T q$ должно вернуть значение, записанное далеко назад - сигнал одной пары против помехи от всех остальных записей

Степень сжатия

$$S_t^\top q_i = \underbrace{(k_i^\top q_i) v_i}_{\text{сигнал}} + \underbrace{\sum_{j \neq i} (k_j^\top q_i) v_j}_{\text{помеха: растёт с числом записей}}$$

байты состояния на слой

линейный слой

1 MiB = 524,288 чисел fp16

softmax-внимание, 128k токенов

512 MiB

×512: теоретико-информационная граница, не задача оптимизации

Гибриды

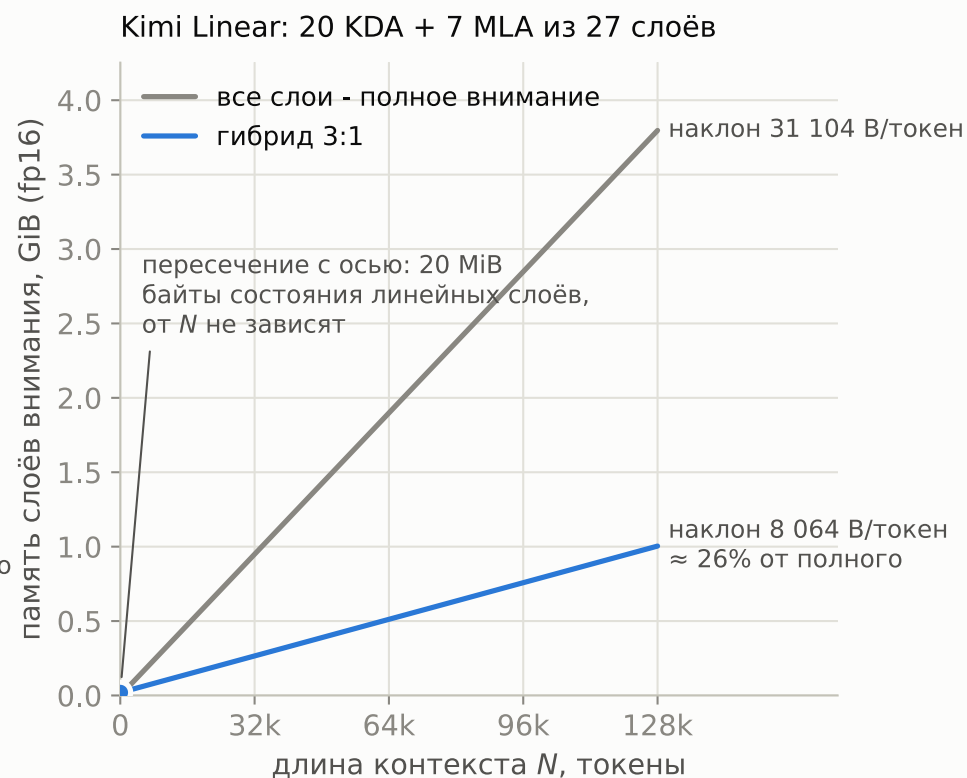
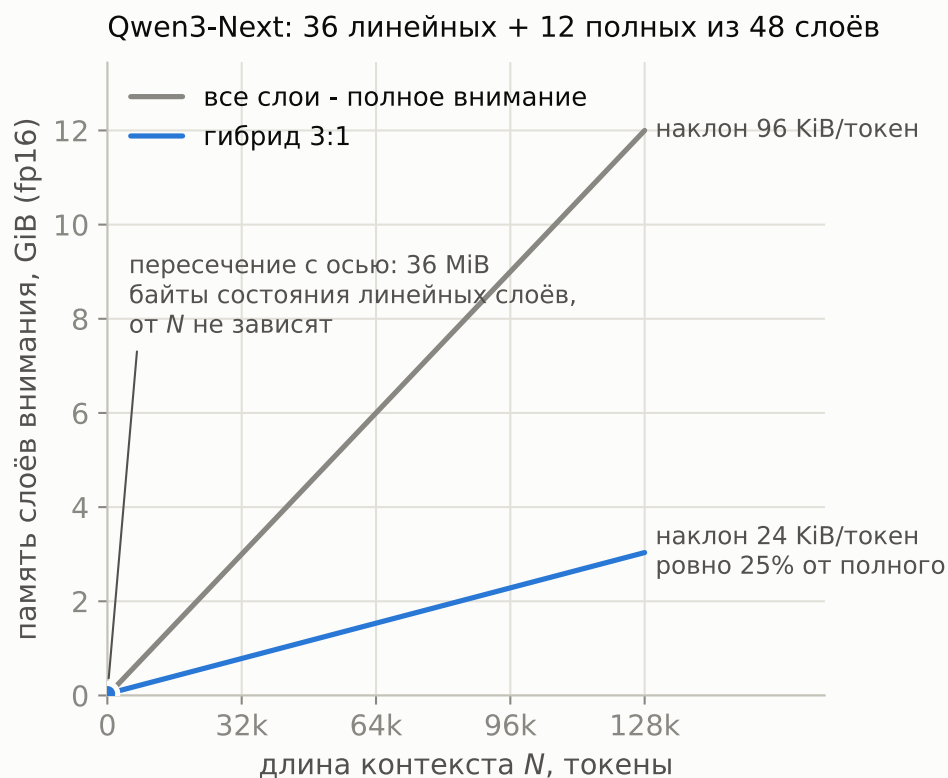
модель	слоёв	линейных	полного внимания	соотношение
Qwen3-Next	48	36	12, каждый четвёртый	ровно 3:1
Kimi Linear, статья				3:1
Kimi Linear, чекпоинт 48B	27	20	7	почти 3:1

в основном линейные слои; немногие слои полного внимания берут на себя точное воспроизведение

Qwen3-Next: две величины

$$\frac{12 \cdot 2 \text{ KiB}}{48 \cdot 2 \text{ KiB}} = \frac{24 \text{ KiB}}{96 \text{ KiB}} = 25\%$$

слой полного внимания, 2 KV-головы шириной 256:
 $2 \cdot 2 \cdot 256 \cdot 2$ байт = 2 KiB на токен
36 линейных слоёв по 1 MiB: 36 MiB, прирост 0



прирост на токен - наклон; байты состояния - пересечение с осью: две величины в разных единицах, одна прямая

Kimi Linear: MLA

MLA кэширует один сжатый латентный вектор на токен в слое: 576 чисел, общий для всех 32 голов

$$576 \cdot 2 = 1152 \quad \text{байта на токен, на слой — от числа голов не зависит}$$

формула $2 \cdot n_{kv_heads} \cdot head_dim$ к MLA не применима

$$1 - \frac{7 \cdot 1152}{27 \cdot 1152} = 1 - \frac{8064}{31104} > 74\% \quad \text{7 слоёв MLA против 27: «до 75%», а не 75\%}$$

Раздел 7

Итог

Когда выбирать линейное внимание

Выбирайте

- длинный контекст, платите за пропускную способность декодирования
- память карты заполняет KV-кэш, а не веса
- много последовательностей одновременно: кэш и выигрыш — на последовательность
- можно гибридизировать

Не выбирайте

- нужно точное воспроизведение: поиск по длинным документам, код с далёкими ссылками, транскрипты агентских сессий
- короткий контекст: ниже нескольких тысяч токенов softmax-декодирование дешевле
- малый бюджет предобучения
- правило без опубликованного чанкового алгоритма

**В основном линейное, а softmax — там, где
живёт точное воспроизведение**

не «линейное вместо softmax»

Открытые проблемы

Ёмкость на байт состояния

- декаплинг стирания и записи в Gated DeltaNet-2

Чанковые алгоритмы для новых правил

- каждому правилу — свой, по отдельной статье

Соотношение и размещение слоёв полного внимания

- Kimi Linear: абляция — 3:1, поставляемая модель — 20:7

Обучение на этапе инференса (test-time training)

- состояние как модель, которую подгоняют онлайн: больше, чем интерпретация?

Литература

- «Transformers are RNNs»: исходная конструкция
 - GLA: гейтирование
 - Mamba2: двойственность пространства состояний
 - чанковый DeltaNet: arXiv 2406.06484
- Gated DeltaNet и Gated DeltaNet-2
 - Kimi Linear: промышленный гибрид
 - обзор эффективного внимания, 2025
 - репозиторий flash-linear-attention: ядра

полные ссылки — в справочнике по архитектурам, [architecture-reference.pdf](#)